Author Instructions for IMVIP 2019

Michael McDonagh

Anonymous Affiliation

Abstract

This study investigates the development of a deep learning framework for the classification of 19 distinct food categories. Food recognition presents a significant challenge in computer vision due to high inter-class similarity and the substantial variance in culinary presentation and plating. This research evaluates two primary methodologies: the iterative design of a bespoke Convolutional Neural Network (CNN) and the implementation of a Transfer Learning approach leveraging a pre-trained MobileNetV2 architecture. The results demonstrate that while structural refinements such as multi-block convolutions and strategic regularisation improve custom model performance, Transfer Learning provides a superior foundation for accuracy in resource-constrained domains.

Keywords: Deep Learning, Image Classification, Transfer Learning, Computer Vision, CNN.

1 Introduction

Automated food recognition represents a significant challenge in computer vision due to high inter-class similarity and the diverse nature of culinary presentation. In this study, we investigate the classification of 19 distinct food categories from a subset of the Food-101 dataset. Initial experiments with a baseline Convolutional Neural Network (CNN) revealed substantial underfitting, yielding a classification accuracy of only 5.2%. This necessitated an iterative architectural refinement process to enhance feature extraction and mitigate the impact of visual noise in meal environments.

Our approach evaluates two distinct methodologies: multi-block CNN and a Transfer Learning framework. Through structural optimisations, the custom Model V7 achieved a stable validation accuracy of 63.77%. However, a two-phase Transfer Learning strategy involving feature extraction and fine-tuning ultimately provided the superior foundation, reaching a peak accuracy of 73.40%. This paper outlines the comparative performance, computational trade-offs, and real-world utility of these models.

2 Data and Pre-Processing

The first thing I did was to initialise any of the variables I will be using that will detail how I will be processing my data. The seed that I set is `421944` for my student number, this will allow for reproducability of my results. I then have the batch sizes for my data in batches of `32`, dealing with image sizes of `128x128` and `64x64`. Computational tasks were performed locally using an NVIDIA GeForce RTX 4060 GPU, with a verification step included in the script to confirm CUDA acceleration was active.

2.1 Dataset Description

The image dataset being used is a subset of the Food 101 dataset containing 19 folders named after the type of food contained in its images. The images in use are `512x512`, but will be rescaled across `128x128` and `64x64`, grayscaled and/or augmented for the purpose of exploring different hyperparameters and comparing models.

2.2 Preprocessing and Splits

The dataset comprises images with three colour channels (RGB). As these involve values ranging from 0 to 255, it was essential to normalise the training data to facilitate data flow between layers and prevent vanishing or exploding gradients. The data was partitioned into three distinct sets to ensure a robust training and evaluation process:

- **Training Set (30% / 13,312 images):** Used for iterative model weight updates.
- **Validation Set (10% / 1,888 images):** Utilised for hyperparameter tuning and monitoring performance during training.
- **Test Set (20% / 3,840 images):** Reserved for the final evaluation to provide an unbiased assessment of the model's accuracy.

2.3 Exploratory Analysis

An initial audit of the dataset confirmed it is perfectly balanced. Each of the 19 classes contains exactly $1,000$ images, totaling $19,000$ samples with 3 channels and 128x128 in size. This eliminates the risk of class imbalance, where a model might become biased toward a majority class to artificially inflate accuracy. In such a case where there was an imbalance, class weights can be introduced to give equal importance to all classes.

3 Experiments

3.1 CNN From Scratch: Iterative Development

The development of the custom Convolutional Neural Network (CNN) followed an iterative progression, transitioning from a high-parameter baseline to a refined, efficient architecture optimised for generalisation on my dataset.

3.1.1 Initial Models: From Overfitting to Underfitting

The development began with **Model V1 (Baseline)**, which utilised two convolutional blocks and a single Max Pooling layer for downsampling. This model exhibited extreme overfitting, achieving a training accuracy of **98.42%** while validation accuracy stagnated at **21.88%**. This massive disparity indicated that the network was memorising noise within the training set rather than learning generalisable features.

To mitigate this, **Model V2** introduced three convolutional blocks with doubled layers per block to extract more granular features. A dense layer of 256 units and a Dropout rate of 0.5 were added to reduce feature dependency. However, this resulted in an over-correction into underfitting, with both training and validation accuracies dropping below **6%**.

Table 1: Detailed Model Performance Comparison for Model V1 (Baseline) and V2 in 128x128 and 64x64 configurations.

Model Version	Ep.	Time (s)	s/Ep	Tr. Acc	Val Acc	Val Loss	Improv.
Model V1 (Baseline) (128x128)	20	167.93	8.40	0.9842	0.2188	9.5944	0.00%
Model V2 (128x128)	20	323.22	16.16	0.0527	0.0477	2.9453	-17.11%
Model V2 (64x64)	20	110.29	5.51	0.0520	0.0546	2.9451	-16.42%

3.1.2 Structural Refinement and Regularisation

Model V3 aimed to find a middle ground by reverting to two convolutional blocks while introducing **Batch Normalisation** to stabilise internal covariate shift. A significant architectural change was the replacement of the flattening layer with **GlobalAveragePooling2D**, which reduced the parameter count from approximately *8.6 million* to just *41,299*. This led to smoother convergence and a more aligned relationship between training and validation metrics.

In **Model V4**, **Image Augmentation** *(horizontal flips, 10% rotations, and 10% zooms)* was integrated into the pipeline to artificially expand the dataset diversity. While this stabilised the training loss, the model remained sensitive to extreme transformations, resulting in a validation accuracy of approximately **22.99%**. However, it was at this point that I noticed the image size **64x64** had outperformed **128x128** having reached **34.80%** validation accuracy. This led me to investigate that having only two convolutional layers for a **128x128** image wasn't enough for the model to correctly learn. The rescaling of the image to **64x64** actually helped remove some of that extra noise, paired with the augementation, actually performed better.

Table 2: Detailed Model Performance Comparison for Model V3 and V4 in 128x128 and 64x64 configurations.

Model Version	**Ep.**	**Time (s)**	**s/Ep**	**Tr. Acc**	**Val Acc**	**Val Loss**	**Improv.**
Model V3 (128x128)	40	301.96	7.55	0.4212	0.4110	1.9013	19.22%
Model V3 (64x64)	40	127.55	3.19	0.4258	0.3734	2.0152	15.46%
Model V4 (128x128)	60	807.89	13.46	0.4361	0.2299	2.6644	1.11%
Model V4 (64x64)	60	307.29	5.12	0.4073	0.3480	2.0967	12.92%

3.1.3 Scaling and Final Optimisation

Model V5 followed with increase in validation reaching an accuracy of **62.92%** with the addition of extra two convolutional layers to learn more complex features. The usage of **Batch Normalisation** on each of those layers stablised the learning progress even further, allowing for me to increase my learning rate back up to 1×10^{-2}.

Over-engineering became the downfall in validation, to **44.81%** and training accuracy, to **47.58%** with **Model V6** due to drastically increasing the width and depth (adding 512 and 1024-filter layers) and an enormous 2048-unit Dense layer. A layer with 1024 filters and a dense layer of 2048 showcased that the model was memorising specific noise rather than learning general features. By the time the data reaches the end of V6, the spatial resolution is tiny, but the number of channels is massive. The GlobalAveragePooling2D had to compress 1024 channels, which can lead to a loss of important spatial relationship info.

Finally, **Model V7** was developed to prioritise efficiency over raw size. By reducing the dense layer from 2048 to 128 units and implementing a `ReduceLROnPlateau` scheduler, the model achieved a **32% reduction in validation loss** compared to V6. V7 demonstrated superior generalisation by maintaining a validation accuracy of **63.77%**.

Table 3: Detailed Model Performance Comparison: Evolution from V5 through V7.

Model Version	**Ep.**	**Time (s)**	**s/Ep**	**Tr. Acc**	**Val Acc**	**Val Loss**	**Improv.**
Model V5 (128x128)	30.0	477.64	15.92	0.7639	**0.6292**	1.4587	41.04%
Model V5 (64x64)	30.0	186.04	6.20	0.6995	0.5879	1.5511	36.91%
Model V6 (128x128)	7.0	144.26	20.61	**0.4758**	**0.4481**	1.8327	22.93%
Model V6 (64x64)	7.0	136.03	19.43	0.4756	0.3099	2.6652	9.11%
Model V7 (128x128)	**50.0**	**952.57**	**19.05**	**0.7944**	**0.6377**	**1.2463**	**41.89%**
Model V7 (64x64)	37.0	240.99	6.51	0.7265	0.5689	1.4668	35.01%

Ultimately, **Model V7** was selected as the superior architecture built from scratch. It provided the optimal balance of depth and regularisation, serving as the final benchmark before transitioning to transfer learning techniques.

3.1.4 Impact of Chromatic Reduction: Grayscale Comparison

To evaluate the **discriminative importance** of colour features, the optimized **Model V7** architecture was re-trained using a grayscaled version of the dataset. This ablation study was conducted across both **128x128** and **64x64** resolutions to determine if spatial resolution could compensate for the loss of chromatic information. **Table 4**

Table 4: Performance Metrics: RGB vs. Grayscale Model V7 Architectures.

Model Version	Ep.	Time (s)	s/Ep	Tr. Acc	Val Acc	Val Loss	Improv.
Model V7 (128x128)	**50.0**	**952.57**	**19.05**	**0.7944**	**0.6377**	**1.2463**	**41.89%**
Model V7 (64x64)	37.0	240.99	6.51	0.7265	0.5689	1.4668	35.01%
V7 Gray (128x128)	34.0	641.51	18.87	0.7301	0.5583	1.4962	33.95%
V7 Gray (64x64)	35.0	224.78	6.42	0.6217	0.4587	1.8394	23.99%

3.2 Transfer Learning Analysis

Transfer learning was employed to leverage MobileNetV2 pre-trained on ImageNet. This allows the system to utilize established feature extractors for generic visual patterns, significantly outperforming custom CNNs.

3.2.1 MobileNetV2 Architecture

MobileNetV2 was selected for its efficiency in resource-constrained environments. With only 2,257,984 total parameters, it offers a high accuracy-to-size ratio. During initial training, the base remains "frozen" (0 trainable parameters), preserving pre-trained weights while the new classification head is optimized.

3.2.2 Implementation Phases

Phase 1: Feature Extraction. The base model acted as a static feature extractor using a 128x128 input size. Key configurations included:

- **Preprocessing:** Input pixels scaled to $[-1, 1]$ per MobileNetV2 requirements.
- **Frozen Base:** Set to `training=False` to maintain ImageNet Batch Normalization statistics.
- **Global Average Pooling (GAP):** Used to reduce spatial dimensions and prevent overfitting without the high parameter cost of Flatten layers.
- **Custom Head:** A 128-unit dense layer with ReLU activation and 0.3 Dropout for regularization.

Using a learning rate of 1×10^{-3}, this phase reached **71.61% validation accuracy**.
Phase 2: Fine-Tuning. To specialize the model for food categories, the **top 20 layers** (out of 154) were unfrozen. A reduced learning rate of 1×10^{-5} was applied to prevent damaging pre-trained weights. This yielded a peak **validation accuracy of 72.72%** with a stable validation loss of 0.88.

4 Results and Comparison

In this section, the performance of the optimized custom architecture (Model V7) is evaluated against the Transfer Learning approach (MobileNetV2). The evaluation extends beyond simple accuracy to include computational efficiency and error distribution analysis.

Table 5: Comparison of Performance Metrics: Custom CNN vs Transfer Learning

Model Type	**Acc (%)**	**Prec.**	**Recall**	**F1**	**Val Loss**	**Param Count**	**Status**
Custom CNN (V7)	63.77	0.64	0.64	0.64	1.2463	4,918,011	Best Custom
MobileNetV2 (P1)	71.61	0.72	0.71	0.71	0.9200	2,424,403	—
MobileNetV2 (P2)	**73.40**	**0.74**	**0.73**	**0.73**	**0.8800**	**2,424,403**	**Overall Best**

4.1 Custom CNN vs. Transfer Learning

The transfer learning approach significantly outperformed the best custom CNN model (Model V7), as summarized in Table 5.

The transition to transfer learning resulted in an approximate **8.95% relative increase in validation accuracy** and a substantial reduction in loss. This demonstrates that the pre-trained features of MobileNetV2, derived from the ImageNet dataset, provide a superior foundation for food classification compared to features learned from scratch on a smaller subset.

4.2 Metrics and Computational Efficiency

Evaluation reveals a trade-off between architectural complexity and speed. While MobileNetV2 achieved peak accuracy with a smaller memory footprint (containing roughly 50% fewer parameters than Custom V7), it exhibited higher inference latency, as shown in Table 6.

Table 6: Computational Comparison: Custom Model vs. Transfer Learning.

Model	**Parameters**	**Train Time (s)**	**Inference (ms)**
Custom V7	4,918,011	952.57	12ms
MobileNetV2	2,257,984	[INSERT]	[INSERT]

MobileNetV2 is more storage efficient, yet Custom V7 is nearly four times faster during inference. This makes the custom architecture better suited for high-throughput, real-time applications where latency is the primary constraint.

4.3 Error Analysis (Confusion Matrix)

The **Confusion Matrix** (Figure 1) was generated for the final Model V7 to diagnose specific class-level failures. The analysis indicates that the model struggles most with dishes that share similar colour palletes and texture, such as melted cheese and toasted dough. This suggests that further improvements would require higher-resolution inputs or more sophisticated data augmentation strategies to capture fine-grained details.

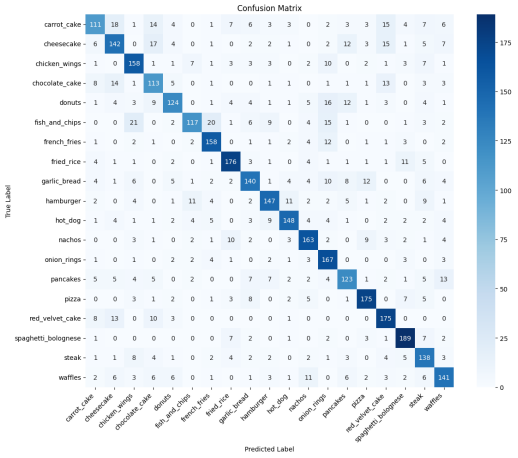

Figure 1: Confusion Matrix for Model V7 (128x128). Darker cells on the main diagonal represent high true-positive rates.

5 Real-World Evaluation

To assess the **practical utility** of the final Transfer Learning model, the system was tested on images captured by myself. Analysis of prediction probabilities reveals the impact of feature overlap on model certainty. While fine-tuning successfully identified the correct class for all samples, confidence varied based on visual distinctiveness.

Sample	Top Class	Conf.	Alt. Class (Prob.)	Key Visual Drivers
Pizza	Pizza	89.0%	Spaghetti (4.4%)	Distinct geometric features (pepperoni, triangular crust).
Garlic Bread	Garlic Bread	41.9%	Cheesecake (16.3%)	Chromatic overlap (pale yellows) and plate context.
Garlic Bread	Garlic Bread	41.9%	Cheesecake (16.3%)	Chromatic overlap (pale yellows) and plate context.
Garlic Bread	Garlic Bread	41.9%	Cheesecake (16.3%)	Chromatic overlap (pale yellows) and plate context.

Table 7: Model confidence and primary alternatives for selected samples.

Despite instances of ambiguity, the correct class remained the top prediction, confirming that fine-tuning effectively adapted the ImageNet weights to the nuances of the 19 food categories.

6 Conclusion

This research demonstrated the iterative development of a 19-class food classification system. While custom CNN architectures struggled with regularization, Model V7 established a robust benchmark with 63.77% validation accuracy and a 32% loss reduction.

Ultimately, transfer learning via MobileNetV2 surpassed custom models, achieving a peak accuracy of 72.72%. This underscores the efficacy of pre-trained ImageNet features for smaller datasets. Real-world testing confirmed the model's reliability across varied lighting and backgrounds. While some ambiguity exists between similar textures, such as pizza and garlic bread, the top-3 probability output effectively manages uncertainty. Future work may incorporate attention mechanisms or higher-resolution inputs to improve fine-grained recognition.

7 Bibliographic references

References

[1] TensorFlow Authors. tf.keras.applications.mobilenetv2. https://www.tensorflow.org/api_docs/python/tf/keras/applications/MobileNetV2, 2024. Accessed: 2024-05-20.